



LANG DİLİ

DİĞER DİLLERDEN AYRILAN ÖZELLİKLERİ

Google bünyesinde K. Thompson ve R. Pike tarafından geliştirilen Go'nun asıl ayırt edici gücü, bünyesine eklediği özelliklerde değil, kasıtlı olarak dışarıda bıraktığı karmaşıklıklarda gizlidir. Google mühendislerinin devasa C++ projelerindeki saatler süren derleme krizine bir reaksiyon olarak doğan Go, *syntax sugar*'ı (örn: *ternary*, *overloading*) reddederek anahtar kelime sayısını 25'te sabitlemiştir. Ayrıca, harici bir kütüphaneye veya *runtime* bağımlılığına ihtiyaç duymadan dilin çekirdeğinde yerel olarak çalışan eşzamanlılık motoru onu bulut sistemler için eşsiz kılmaktadır. Rob Pike bu tasarımı şu sözlerle özetler:

"Sadelik, karmaşıklığı gizleme sanatıdır. Go karmaşıktır, ancak basit görünür. Bu durum derin bir tasarım ve uzun bir emek gerektirmiştir. Bu dil, büyük ekiplerde en verimli ve bakım maliyeti en düşük kod tabanını inşa etmesini hedefler".

Diğer özellikleri de sonraki paragraflarda inceleyelim.

NESNEYE YÖNELİK KAVRAMLAR VE KAPSÜLLEME

Golang, kısmen bir *OOP* dilidir ama alışlagelmiş *class* ve *inheritance* mekanizmaları bulunmuyor. Go, veriyi ve davranışı bütünleşik bir yapıda sunan dillerin aksine, bu iki kavramı mimari olarak kesin bir şekilde birbirinden ayırır.

Özellikleri tanımlamak için *struct* bloklarını kullanan dil, davranışı ise tamamen bağımsız *interface* yapılarına teslim eder. Bu yaklaşım, OOP terimini ortaya atan Alan Kay'ın "OOP'nin özü katı sınıf ağaçları değil; mesajlaşma, yerel saklama ve durumun gizlenmesidir" vizyonu ile kusursuz bir şekilde örtüşür. Go, polimorfizmi açıkça *implements* zorunlu kılmadan, çalışma zamanında örtük olarak yürütür ve *Duck Typing* olarak adlandırılır. Eğer bir yapı, arayüzün talep ettiği metotlara sahipse, derleyici onu otomatik olarak o arayüzün bir üyesi kabul eder.

Kapsülleme ise anahtar kelimelerin karmaşasından kurtarılarak doğrudan paket düzeyinde sözdizimsel bir kurala bağlanmıştır. Bir tanımlayıcının ilk harfi büyükse herkes erişebilir (*public*), küçükse paket dışına kapatılır (*private*).

```
package main

type Message struct {
    Data    []byte
    Timestamp string
}

type Finder interface {
    Find(word string) ([]Message, error)
}

type TwitterSource struct { Username string }
type SkypeSource    struct { login    string }
```



EŞZAMANLILIK VE PARALELLİK

Go dili, *eşzamanlılık* ve mesaj geçişi kavramlarını *CSP* (Communicating Sequential Processes) teorisine dayanarak dilin çekirdeğine entegre etmiştir. Go'da, megabaytlarca bellek tüketen ağır işletim sistemi *threads* yerine, yalnızca ~2 KB başlangıç yığın belleğine sahip, çalışma zamanı tarafından yönetilen hafif *Goroutine* mimarisini kullanır.

go anahtar kelimesiyle tetiklenen binlerce goroutine, arka planda çalışan *M:N Scheduler*

vasıtasıyla gerçek fiziksel thread'ler üzerine akıllıca çoklanır. Böylece bağlam geçişleri kullanıcı alanında (*user-space*) tamamlanır ve çekirdeğe yapılan pahalı sistem çağrıları engellenir. Ağ işlemlerinde ise *Netpoller* mekanizması asenkron *epoll/kqueue* havuzlarını kullanarak thread bloklamalarının önüne geçer.

Go, paylaşılan bellek alanları ve kilit (*mutex/semaphore*) karmaşası yerine, verinin Kanallar üzerinden güvenli aktarımı prensibini benimser: "Belleği paylaşarak iletişim kurma; iletişim kurarak belleği paylaş." Kanaldan veri okuma ve yazma işlemleri varsayılan olarak bloklayıcıdır, bu da ek bir senkronizasyon mekanizmasına olan ihtiyacı ortadan kaldırır. Bu yaklaşım test altyapısına da yansır: *-p*, *-parallel* ve *t.Parallel()* kombinasyonları, testlerin CPU çekirdeklerini sonuna kadar sömürerek eşzamanlı çalışmasını sağlar.

```
func (t TwitterSource) Find(word string) ([]Message, error) {
    ch := make(chan []Message)
    go func() {
        ch <- []Message{{Data: []byte("Twitter Verisi"), Timestamp: "2006"}}
    }()
    return <-ch, nil
}
```



FONKSİYONEL PROGRAMLAMA YAPILARI

Alt programları *first-class citizen* olarak destekleyen dillerin fonksiyonel paradigmayı güçlendirdiğini belirtir. Go saf fonksiyonel bir dil olmasa da, bu tanıma tam olarak uyuyor.

Bir fonksiyonu sıradan bir değişken gibi atayabilir, başka bir fonksiyona parametre *higher-order function* olarak gönderebilir veya bir işlem sonucunda geriye döndürebiliriz. İsimsiz fonksiyonlar aracılığıyla kurulan Kapatmalar (*Closures*), tanımlandıkları dış kapsamdaki (*lexical scope*) değişkenleri kendi içlerinde hapseder.

Bu sayede değişkenlerin yaşam süreleri, fonksiyonun icrası bitse dahi dinamik olarak korunur.

BINDING VE BELLEK YÖNETİMİ

Tip sisteminde Go son derece katıdır; statik ve güçlü bir tipe sahiptir. Tür bağlamaları derleme zamanında (*early binding*) çözülür, dolayısıyla çalışma zamanında sürpriz veri dönüşüm hatalarına izin verilmez. Dinamik bağlama (*late binding*) ise yalnızca *interface* metot çağrılarında rölantiye alınır.

Bellek dağıtımında derleyici *Escape Analysis* yürütür. Bir değişkenin yaşam süresi mevcut fonksiyonun dışına taşmıyorsa, *stack*'te hızlıca oluşturulur. Eğer bir *pointer* veya referans vasıtasıyla dışarıya sızıyorsa, derleyici nesneyi otomatik olarak *heap* alanına fırlatır. Go'daki en büyük yanılgı, otomatik *Garbage Collector* bellek sızıntılarını tamamen önleyeceğidir. Go'da iki tehlikeli sızıntı türü vardır:

1. **Goroutine:** Bir kanal üzerinde sonsuza kadar bloke olan bir goroutine'i GC asla temizleyemez ve yığın belleği sistemde asılı kalır.



```
func LeakMemory() {
    ch := make(chan int)
    go func() {
        data := <-ch
        fmt.Println(data)
    }()
}
```

2. **Dilim Sızıntıları:** Dilimler (*slices*) *stack*'te sadece 24 baytlık bir tanımlayıcı (*pointer, len, cap*) tutar, veriler ise *heap*'tedir. Devasa bir veriden *small := large[0:4]* şeklinde küçük bir kesit aldığınızda, büyük veri tabanının tamamı Heap bellekte kilitli kalır; çünkü küçük dilim hala kökteki adrese referans vermektedir.

DİL SÖZDİZİMİ VE ANLAMBİLİM ÖZELLİKLERİ

Derleme aşamasında Go anlambilimi (semantics) adeta bir sparta kuralı işletir. Kodda satır sonlarında ; işaretlerini görmezsiniz; çünkü Leksikal Analizör (Scanner) satırları tararken uygun yerlene sanal ; enjekte ederek Sözdizimi Analizörünün (Parser) yükünü hafifletir.

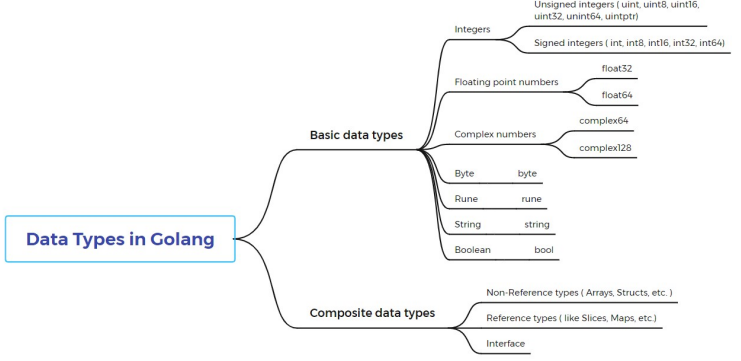
Semantik olarak Go asla tolerans göstermez: Kullanılmayan bir değişken veya atıl bir import uyarı değil, doğrudan Derleme Hatasıdır. Akış kontrolünde ise try-catch istisna mimarisi tamamen reddedilmiştir; hatalar fonksiyonların döndürdüğü sıradan veri değerleridir. İstisnai durumlarda ise panic ve recover mekanizmaları devreye girer; ancak bunlar iş mantığı için değil, sadece sistemi çöküşten kurtarmak için rölantiye alınır.

Mantıksal operatörlerdeki Kısa Devre (Short-Circuit Evaluation) mantığına da dikkat edilmelidir. Sol taraf sonucu belirlediğinde sağ taraftaki fonksiyon tetiklenmez; eğer o fonksiyonun içinde küresel bir durumu değiştirme gibi bir yan etki varsa, çalışma zamanında tespit edilmesi imkansız mantıksal açıklar doğabilir.

ADT YAPISI, VERİ TÜRLERİ VE GENERICLİK

Soyut Veri Tipleri (ADT) Go'da struct ve metot hiyerarşileriyle kurulur. Çoklu kalıtım hatalarını (Diamond Problem) engellemek adına Struct Embedding (Yapı Gömme/ Bileşim) mimarisi uygulanır.

Bellek güvenliğinde ise "rastgele çöp bellek" kavramı yok edilmiştir. İlkendirilmeyen her değişken otomatik olarak bir Zero-Value (0, "", false, nil) değeriyle eşleşir.



Dilin en zayıf semantik halkası yerleşik bir enum tipinin olmamasıdır; taklit iota ile yapılır ancak çalışma zamanında bu tipe aralık dışı geçersiz bir tam sayı atanabilir ve derleyici bunu engelleyemez. Parametrik polimorfizm (Generics) ise tip güvenliğinden ve hızdan ödün vermeden type Stack[T any] struct { ... } sözdizimi ve any kısıtlayıcıları ile ekosisteme dahil edilmiştir.

```
type Sources []Finder
type Person struct {
    FullName string
    Sources // Anonim Gömme (Kalıtım yerine Bileşim)
}
type Stack[T any] struct { elements []T } // Generics
```

Programlama Dilleri Ödev Raporu için
Madina Yusupova 24360859922 tarafından hazırlandı
Video Linki:

Kaynakçalar:

- <https://tproger.ru/translations/chto-delaet-go-takim-neobychnym>
- <https://habr.com/ru/articles/243593/>
- <https://yteblog.bilgem.tubitak.gov.tr/post-assets/presentations/2022/12/golang.pdf>
- <https://www.slideshare.net/slideshow/golang-book-go-programlama-dili-temelleri/252012298>
- <https://habr.com/ru/articles/908396/>
- Concepts of programming languages, Robert, W. Sebesta, Pearson, 11. Versiyon, 2016